Contents

Software Requirements Specification (SRS)

LeadRadar AI — Agentic Client-Discovery System

Version: 0.2 (MVP Draft) **Status:** Living document — updated as checkpoints are completed **Build approach:** Free-tier-first, agentic architecture, built incrementally via Claude Code checkpoints

---

1. Introduction

1.1 Purpose

LeadRadar AI is an agentic system that discovers local businesses likely to need digital services (websites, apps, automation), audits their existing digital presence, scores them as leads, and (in later phases) generates outreach and tracks them through a CRM pipeline.

This SRS defines the MVP scope, system architecture, and a **checkpoint-based build plan** intended to be executed with Claude Code, one checkpoint at a time.

1.2 Scope (MVP — Phase 1)

> "Find restaurants in one Mumbai neighborhood that have no website or a poor one, and explain why."

Phase 1 deliberately excludes CRM, outreach generation, multi-vertical support, and multi-city support. Those are Phase 2+ and are listed in Section 8 for context only.

1.3 Definitions

Term	Meaning
Agent	An LLM loop that decides which tool to call next based on a goal and prior results
Provider	An LLM API service (Groq, Gemini, OpenRouter, etc.)
Router	The subsystem that picks a provider/model and fails over on quota/error

Term	Meaning
Lead	A business record enriched with audit + score data
Checkpoint	A self-contained build milestone with its own Claude Code prompt, CLAUDE.md update, and test step

1.4 Guiding Principle

Every checkpoint must produce something **runnable and verifiable** before the next one starts. No checkpoint should touch more than one architectural layer at a time.

---

2. Overall Description

2.1 Product Perspective

LeadRadar AI is a standalone agentic backend (Phase 1 has no frontend — CLI/JSON output only) that will later be wrapped in a Next.js dashboard once the core discovery→audit loop is proven reliable.

2.2 User Classes

- **Phase 1:** Developer only (you), running scripts locally to validate the concept.
- **Phase 2+:** Freelancers/agencies using a dashboard.

2.3 Operating Constraints

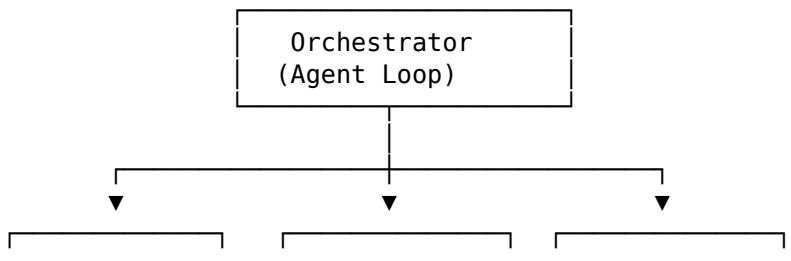
- Must run entirely on free-tier APIs during MVP.
- Must not depend on any single LLM provider being available (quota exhaustion is expected, not exceptional).
- Must respect ToS of all data sources — no scraping of platforms that explicitly forbid it (LinkedIn, Justdial, etc. excluded from Phase 1).

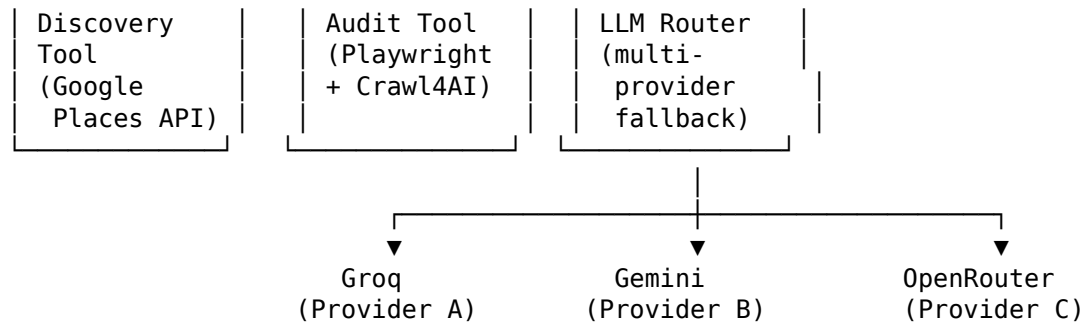
2.4 Assumptions

- Google Places API free monthly credit is sufficient for MVP-scale queries (~20–50 businesses per test run).
- At least one of Groq / Gemini / OpenRouter free tier is available at any given time.

---

3. System Architecture Overview





The **LLM Router** is a first-class subsystem, not a utility function — every agent/tool call that needs an LLM goes through it, never calls a provider SDK directly.

---

4. Functional Requirements

FR-1: Business Discovery

- FR-1.1: System shall accept a location (lat/lng or place name) + radius + business category.
- FR-1.2: System shall query Google Places API and return name, address, website (nullable), rating, phone.
- FR-1.3: System shall deduplicate results by place_id.

FR-2: Website Audit

- FR-2.1: If `website` is null → flag as `NO_WEBSITE` immediately, skip further audit.
- FR-2.2: If `website` exists → fetch via Playwright, capture desktop screenshot, check load success/failure/timeout.
- FR-2.3: System shall extract basic page signals: has SSL, mobile viewport meta tag present, page load time.
- FR-2.4: System shall send screenshot + signals to the LLM Router for a qualitative verdict (2–3 sentence explanation + needs-redesign boolean).

FR-3: LLM Provider Router (multi-key fallback)

- FR-3.1: System shall maintain an ordered list of providers, each with its own API key(s).
- FR-3.2: On a request, router shall attempt providers in priority order.
- FR-3.3: On quota-exceeded (HTTP 429) or auth error, router shall mark that provider "cooling down" and retry the next one **within the same request** — the caller should never see the failure.
- FR-3.4: Cooldown duration shall be configurable per provider (default: 60 minutes for 429s).
- FR-3.5: Router shall log every fallback event (provider, reason, timestamp) for later review.
- FR-3.6: Router shall support multiple keys **per provider** (e.g., 3 Gemini keys), rotating within a provider before falling to the next provider.
- FR-3.7: Config (provider list, keys, priority, cooldown) shall live in a single config file/env block, not hardcoded.

FR-4: Lead Scoring (MVP-simplified)

- FR-4.1: System shall compute a basic score from: no-website (+40), poor audit verdict (+30), low rating (+10), no SSL (+10), slow load (+10).
- FR-4.2: Score bucket: 70+ = HOT, 40–69 = WARM, <40 = COLD.
- Full multi-factor scoring (funding, hiring signals, etc.) is deferred to Phase 2.

FR-5: Output

- FR-5.1: System shall output results as a JSON file (Phase 1) with all discovery + audit + score fields per business.
- FR-5.2: System shall print a human-readable summary table to console.

---

5. Non-Functional Requirements

ID	Requirement
NFR-1	Must run fully on free-tier services during Phase 1
NFR-2	A single provider quota failure must never crash a run
NFR-3	Full run of 20 businesses should complete in under 5 minutes
NFR-4	All API keys stored in `.env`, never committed to git
NFR-5	System must be resumable — if it crashes mid-run, already-processed businesses aren't reprocessed
NFR-6	Code must be modular enough that Phase 2 (multi-vertical, multi-city, CRM) doesn't require rewriting Phase 1 core

---

6. External Interfaces

Service	Purpose	Tier
Google Places API	Business discovery	Free monthly credit
Groq API	Primary LLM provider (fast)	Free tier
Gemini API	Fallback LLM provider	Free tier
OpenRouter	Secondary fallback / testing	Free-tier models
Playwright (self-hosted)	Screenshots, page load checks	Free (local)
Crawl4AI (self-hosted)	Page content extraction	Free (local)

---

7. LLM Router — Design Detail

```
PROVIDERS = [
    {"name": "groq", "keys": ["GROQ_KEY_1", "GROQ_KEY_2"], "model": "llama-3.3-70b"},
```

```
    {"name": "gemini", "keys": ["GEMINI_KEY_1"], "model": "gemini-2.0-flash"},
    {"name": "openrouter", "keys": ["OPENROUTER_KEY_1"], "model": "some-free-model"},
]
```

Router behavior: 1. Try `providers[0]`, `keys[0]`. 2. On 429/auth error → mark that key cooling down, try next key in same provider. 3. If all keys for that provider are cooling down → move to next provider. 4. If **all** providers exhausted → raise a clear error (don't silently fail) so the orchestrator can pause the run rather than produce garbage output. 5. Cooldown state stored in-memory for Phase 1 (a simple dict), moved to Redis in Phase 2 once multiple workers exist.

This is exactly the piece worth building carefully first — it's the thing that keeps your whole system alive on free tiers.

---

8. Phase 2+ (Deferred — Not in MVP)

Multi-vertical + multi-city discovery, full lead scoring, outreach generator, proposal generator, CRM pipeline, dashboard UI, Chrome extension, team workspace, integrations (Gmail/Slack/Zapier), premium features. Reference the original product brief for full detail — nothing here changes; it's simply out of scope until Phase 1 is proven.

---

9. Project Structure (Enterprise-Grade)

This structure follows patterns used in production FastAPI systems (feature-based/domain-driven organization, inspired by Netflix Dispatch and the widely-adopted `fastapi-best-practices` conventions), extended with a dedicated **agentic layer** (`agents/`, `llm/`, `tools/`) since that's the actual core of this product — most FastAPI boilerplates don't need to plan for this, LeadRadar does.

Design principle: routers stay thin (HTTP only) → services hold business logic → repositories hold DB queries → agents/tools are called *by* services, never by routers directly. The LLM Router is isolated in its own package so nothing else in the codebase talks to a provider SDK directly.

```
leadradar-ai/
├── .github/
│   └── workflows/
│       ├── ci.yml                  # lint, type-check, test on every PR
│       └── deploy.yml
│
├── alembic/                        # DB migrations
│   ├── versions/
│   └── env.py
│
├── src/
│   └── leadradar/
│       ├── __init__.py
│       ├── main.py                 # FastAPI app entrypoint, mounts routers
│       │
│       ├── core/                   # cross-cutting infrastructure
│       │   ├── config.py           # pydantic Settings — loads .env, typed
│       │   ├── logging.py          # structured logging setup
│       │   ├── security.py         # auth/secrets helpers (Phase 2+)
```

```
│       │   ├── exceptions.py         # global exception handlers
│       │   └── constants.py
│       │
│       ├── db/
│       │   ├── base.py               # SQLAlchemy declarative base
│       │   ├── session.py            # engine + session factory
│       │   └── models/               # ORM models, one file per domain
│       │       ├── business.py
│       │       ├── lead.py
│       │       └── user.py
│       │
│       ├── agents/                   # ── AGENTIC LAYER (core of the product) ──
│       │   ├── orchestrator.py       # top-level agent loop / decision logic
│       │   ├── discovery_agent.py
│       │   ├── audit_agent.py
│       │   ├── scoring_agent.py
│       │   ├── outreach_agent.py     # Phase 2
│       │   └── prompts/            # versioned prompt templates (not inline strings)
│       │       ├── audit_verdict.md
│       │       └── outreach_email.md
│       │
│       ├── llm/                      # ── LLM PROVIDER ROUTER SUBSYSTEM ──
│       │   ├── router.py             # multi-provider fallback logic (Section 7)
│       │   ├── providers/
│       │   │   ├── base.py           # abstract Provider interface
│       │   │   ├── groq_provider.py
│       │   │   ├── gemini_provider.py
│       │   │   └── openrouter_provider.py
│       │   ├── cooldown.py           # per-key/provider cooldown state
│       │   └── schemas.py            # request/response pydantic models
│       │
│       ├── tools/                    # functions agents call (no LLM logic here)
│       │   ├── places_client.py      # Google Places API wrapper
│       │   ├── web_audit.py          # Playwright + Crawl4AI page checks
│       │   ├── screenshot.py
│       │   └── scoring_rules.py      # pure scoring function, FR-4
│       │
│       ├── features/                 # domain modules = REST API layer
│       │   ├── discovery/
│       │   │   ├── router.py         # HTTP endpoints only
│       │   │   ├── schemas.py        # request/response models
│       │   │   ├── service.py        # business logic, calls agents/tools
│       │   │   └── dependencies.py
│       │   ├── leads/
│       │   │   ├── router.py
│       │   │   ├── schemas.py
│       │   │   ├── service.py
│       │   │   └── repository.py     # DB queries isolated here
│       │   ├── crm/                  # Phase 2
│       │   └── outreach/             # Phase 2
│       │
│       ├── workers/                  # background/async jobs
│       │   ├── celery_app.py
│       │   └── tasks/
```

```
│       │       └── run_discovery_job.py
│       │
│       └── shared/
│           ├── utils.py
│           └── types.py
│
├── tests/
│   ├── unit/
│   │   ├── test_llm_router.py
│   │   ├── test_scoring_rules.py
│   │   └── test_web_audit.py
│   ├── integration/
│   │   └── test_discovery_pipeline.py
│   └── conftest.py
│
├── scripts/                          # standalone CLI entrypoints (used in MVP)
│   ├── run_discovery.py
│   └── seed_data.py
│
├── screenshots/                      # gitignored — runtime output
├── output/                           # gitignored — JSON exports
│
├── docker/
│   ├── Dockerfile
│   └── docker-compose.yml
│
├── .env.example
├── .gitignore
├── pyproject.toml
├── CLAUDE.md
├── SRS.md
└── README.md
```

9.1 Why this shape

- **`agents/` and `llm/` are separated from `features/`** — the agentic/LLM logic is the product's core IP and changes independently of the API layer. If you swap FastAPI for something else later, `agents/` and `llm/` don't move.
- **`tools/` contains no LLM calls** — tools are deterministic functions (API calls, scraping, math). Agents *decide* to call tools; tools themselves stay dumb and testable in isolation.
- **Feature folders (`features/discovery`, `features/leads`) mirror the fastapi-best-practices convention** — router → schemas → service → repository, each with a single responsibility, so a new contributor can find anything by domain, not by file type.
- **Prompts live in versioned files (`agents/prompts/*.md`), not inline strings** — makes prompt iteration reviewable in git diffs and testable independently of code.

9.2 Phase 1 (MVP) subset — don't build all of this yet

Only build what the Checkpoint plan in Section 10 actually needs. Everything else above is the target shape to grow into, not a Day 1 requirement.

Phase 1 needs	Maps to
`llm/router.py`, `llm/providers/`, `llm/cooldown.py`	Checkpoint 1
`tools/places_client.py`	Checkpoint 2
`tools/web_audit.py`, `tools/screenshot.py`	Checkpoint 3
`agents/audit_agent.py` (or a flat `verdict.py` for MVP simplicity)	Checkpoint 4
`tools/scoring_rules.py`	Checkpoint 5
`scripts/run_discovery.py`	Wires it all together, Checkpoints 2–5

`db/`, `features/`, `workers/`, `alembic/`, Docker, and CI are **Phase 2** — introduced only once the MVP pipeline (Section 10) works end-to-end and you're wrapping it in a real API + dashboard.

9.3 What Every Folder/File Means and Why It Exists

A plain-English walkthrough of the full tree from Section 9, in the order a new contributor (or future-you) would actually need to understand it.

Root level

Path	Purpose
`.github/workflows/`	GitHub Actions automation. `ci.yml` runs lint/type-check/tests on every pull request so broken code can't merge. `deploy.yml` automates pushing to production. Not needed until you have a team or a CI habit — Phase 2.
`alembic/`	Database migration tool for SQLAlchemy. Every time you change a DB model (add a column, new table), Alembic generates a versioned migration script here so the schema change can be applied/rolled back safely instead of hand-editing the database. Only needed once `db/` exists (Phase 2).
`src/leadradar/`	The actual Python package. Everything importable lives under `src/` — this is a deliberate convention (called the "src layout") that prevents accidentally importing your code from the wrong place during testing.
`tests/`	All automated tests, mirroring the structure of `src/leadradar/` so any file's tests are easy to find.
`scripts/`	Standalone runnable files that aren't part of the importable package — e.g. `run_discovery.py`, which you literally run from the terminal. This is where your **entire Phase 1 MVP entrypoint lives**.

Path	Purpose
`screenshots/`	Where Playwright saves website screenshots at runtime. Gitignored — this is generated output, not source code.
`output/`	Where the final scored lead JSON files land after a run. Also gitignored.
`docker/`	Containerization files so the app can run identically on any machine/server. Only matters once you're deploying (Phase 2).
`.env.example`	A template showing which environment variables the project needs (API keys, DB URL) **without the real secret values**. Real secrets go in a `.env` file that's gitignored and never committed.
`.gitignore`	Tells git which files/folders to never track — `.env`, `screenshots/`, `output/`, `__pycache__/`, etc.
`pyproject.toml`	Modern Python's single config file — declares dependencies, project metadata, and tool settings (pytest, linters) in one place, replacing the old `requirements.txt` + `setup.py` combo.
`CLAUDE.md`	Claude Code's persistent memory file for this project — the running log of what's been built, decisions made, and gotchas discovered, updated after every checkpoint (Section 10).
`SRS.md`	This document, kept in the repo so requirements stay versioned alongside the code.
`README.md`	The first thing anyone (including you, in six months) reads to understand what the project is and how to run it.

`src/leadradar/core/` — infrastructure every other module depends on

File	Purpose
`config.py`	Loads and validates every environment variable (API keys, DB URL, cooldown durations) into a single typed `Settings` object using pydantic. Nothing else in the codebase reads `os.environ` directly — it all goes through this file, so there's one place to see every config value the app needs.
`logging.py`	Sets up structured logging (so log lines are consistent and searchable) once, centrally, instead of every file configuring its own logger differently.
`security.py`	Auth helpers — password hashing, token verification. Empty/unused in Phase 1 (no users yet), becomes relevant once the dashboard has logins in Phase 2.

File	Purpose
`exceptions.py`	Custom exception classes (like `RouterExhaustedError`) and FastAPI's global exception handlers, so errors return consistent, readable responses instead of raw stack traces.
`constants.py`	Fixed values used across the app (score thresholds, default radius, timeout values) that don't belong in `.env` because they're not secrets or per-environment settings — they're just named constants.

`src/leadradar/db/` — everything about talking to the database (Phase 2)

File	Purpose
`base.py`	The SQLAlchemy "declarative base" class that every table model inherits from — a required piece of plumbing SQLAlchemy needs to know your models exist.
`session.py`	Creates the database connection (engine) and manages sessions (a session = one unit of work talking to the DB, opened and closed per request).
`models/business.py`, `models/lead.py`, `models/user.py`	Each file defines one database table as a Python class (ORM model) — `business.py` mirrors the businesses you discover, `lead.py` the scored/enriched version, `user.py` the people using the dashboard. Split by domain so each file stays small and ownership is obvious.

`src/leadradar/agents/` — the agentic layer (this is the product's core)

File	Purpose
`orchestrator.py`	The top-level control loop — decides which agent/tool to invoke next based on the current goal and what's already been done. This is the "brain" that turns a request like "find restaurants in Bandra" into a sequence of tool calls.
`discovery_agent.py`	Wraps the discovery step — decides how to query `tools/places_client.py`, handles pagination/retries at the agent-reasoning level (as opposed to the tool's own retry logic).
`audit_agent.py`	Wraps the website-audit step — orchestrates calling `tools/web_audit.py` and `tools/screenshot.py`, then asks the LLM Router for a verdict.

File	Purpose
`scoring_agent.py`	Wraps `tools/scoring_rules.py` — in Phase 1 this might just call the pure function directly, but as scoring gets more nuanced (e.g. LLM-assisted scoring), this is where that reasoning would live.
`outreach_agent.py`	Phase 2 — will decide tone/channel/content strategy for generating outreach messages per lead.
`prompts/audit_verdict.md`, `prompts/outreach_email.md`	The actual prompt text sent to the LLM, kept in separate files instead of hardcoded strings inside Python. This means you can edit and version a prompt's wording without touching code, and diff prompt changes cleanly in git history.

`src/leadradar/llm/` — the multi-provider router subsystem (Section 7)

File	Purpose
`router.py`	The `Router` class itself — the single entrypoint every other part of the codebase calls when it needs an LLM completion. Implements the try-provider-A, fallback-to-B-on-429 logic from Section 7.
`providers/base.py`	An abstract interface defining what every provider must implement (a `.complete(prompt)` method). This means adding a new provider later (say, a fourth free API) means writing one new file, not touching the router's core logic.
`providers/groq_provider.py`, `gemini_provider.py`, `openrouter_provider.py`	One file per actual LLM API, each translating the router's generic request into that specific provider's API format.
`cooldown.py`	Tracks which API keys are temporarily "resting" after hitting a quota limit, and until when — the router checks this before attempting a key.
`schemas.py`	Pydantic models defining exactly what a request-to-the-router and a response-from-the-router look like, so every caller gets consistent, validated data.

`src/leadradar/tools/` — deterministic functions (no LLM calls live here)

File	Purpose
`places_client.py`	Wraps the Google Places API — takes a location/radius/category, returns a clean list of businesses. Pure API-calling logic, no reasoning.

File	Purpose
`web_audit.py`	Uses Playwright and Crawl4AI to check if a website loads, how fast, whether it has SSL/mobile support. Pure technical checks, no opinions.
`screenshot.py`	Captures and saves a screenshot of a given URL. Single responsibility, reusable by any agent that needs a visual.
`scoring_rules.py`	The pure scoring formula from FR-4 (no-website +40, poor verdict +30, etc.) — deliberately just math, no LLM involved, so it's fast, free, and 100% predictable/testable.

Why tools are separate from agents: tools are simple, deterministic, and unit-testable in isolation (call it with fixed input, assert fixed output). Agents are where the "should I call this tool, and with what" reasoning happens. Keeping them apart means you can test 90% of your logic without ever calling an LLM.

`src/leadradar/features/` — the REST API layer (Phase 2, once there's a dashboard)

File	Purpose
`discovery/router.py`, `leads/router.py`	Define the actual HTTP endpoints (e.g. `POST /discovery/run`) — deliberately thin, just receiving requests and calling a service.
`.../schemas.py`	Pydantic models defining exactly what shape of JSON the API accepts and returns — separate from the DB models in `db/models/`, because what you store and what you expose over HTTP aren't always identical.
`.../service.py`	The actual business logic for that domain — e.g. "run a discovery job" involves calling the discovery agent, saving results, and triggering the audit agent. This is where orchestration-at-the-feature-level happens.
`leads/repository.py`	All database queries for leads isolated in one file, so `service.py` never writes raw SQL/ORM queries inline — makes swapping databases or optimizing queries a one-file change.
`crm/`, `outreach/`	Placeholder domains for Phase 2 features from the original product brief — pipeline tracking and outreach generation.

`src/leadradar/workers/` — background jobs (Phase 2)

File	Purpose
`celery_app.py`	Configures Celery (the task queue) so long-running jobs (like scanning 200 businesses) run in the background instead of blocking an API request.
`tasks/run_discovery_job.py`	The actual background task definition — what Celery executes when a discovery job is queued.

`src/leadradar/shared/` — small reusable pieces that don't belong anywhere else

File	Purpose
`utils.py`	Small generic helper functions (e.g. formatting, hashing) used across multiple domains — deliberately kept minimal so this file doesn't become a dumping ground.
`types.py`	Shared type definitions/aliases used across the codebase.

`tests/`

Folder	Purpose
`unit/`	Fast tests of one function/class in isolation, no real network calls — e.g. `test_scoring_rules.py` just checks the math, `test_llm_router.py` simulates failures without hitting real APIs.
`integration/`	Slower tests that exercise multiple pieces together, e.g. `test_discovery_pipeline.py` running the full discover→audit→score flow against real or lightly-mocked services.
`conftest.py`	Shared pytest fixtures (reusable setup code, like a fake `Settings` object) available to every test file automatically.

---

10. Checkpoint-Based Build Plan (Claude Code)

Rule for every checkpoint: finish the checkpoint → run the test → update `CLAUDE.md` with what was built and any decisions/gotchas → only then move to the next prompt below.

CLAUDE.md starter (create this first, before Checkpoint 1)

```
# LeadRadar AI — Project Memory

## What this project is
Agentic lead-discovery tool. Phase 1 MVP: find restaurants in a Mumbai
```

```
neighborhood with no/poor websites, audit them, score them, output JSON.

## Architecture decisions log
(Claude Code appends a dated entry here after each checkpoint)

## Known gotchas
(append here as discovered)
```

Checkpoint 1 — Project scaffold + LLM Router

Goal: A working multi-provider LLM router, tested in isolation, nothing else.

Claude Code prompt:

```
Set up a Python project called leadradar using this structure (only
create what's needed for this checkpoint, leave placeholders for the rest):

leadradar-ai/
├── src/leadradar/
│   ├── __init__.py
│   ├── core/
│   │   └── config.py        # pydantic Settings, loads .env
│   └── llm/
│       ├── __init__.py
│       ├── router.py        # Router class
│       ├── cooldown.py      # cooldown state tracking
│       ├── schemas.py       # request/response pydantic models
│       └── providers/
│           ├── __init__.py
│           ├── base.py      # abstract Provider interface
│           ├── groq_provider.py
│           ├── gemini_provider.py
│           └── openrouter_provider.py
├── tests/unit/test_llm_router.py
├── .env.example
├── pyproject.toml
└── CLAUDE.md

Requirements:
- pyproject.toml with httpx, python-dotenv, pydantic, pydantic-settings, pytest
- .env.example with placeholders for GROQ_KEY_1, GROQ_KEY_2, GEMINI_KEY_1, OPENROUTER_KEY_1
- core/config.py: pydantic Settings class loading all provider keys from .env
- llm/providers/base.py: abstract Provider class with a .complete(prompt) method
- llm/providers/{groq,gemini,openrouter}_provider.py: concrete implementations
- llm/cooldown.py: in-memory dict tracking which keys are cooling down and until when
- llm/router.py: Router class that:
  - takes a provider config list (name, keys, model)
  - has a .complete(prompt: str) -> str method
  - tries providers/keys in order, catches 429 and auth errors,
   marks that key as cooling down for 60 minutes via cooldown.py, retries next key/provider
  - raises a clear RouterExhaustedError if everything fails
  - logs every fallback event with provider name, reason, timestamp
- tests/unit/test_llm_router.py: pytest test that intentionally simulates
```

```
  a 429 on the first provider to prove fallback works
Do not add any other functionality yet — no agents/, tools/, or features/ folders yet.
```

Test: Run `pytest tests/unit/test_llm_router.py -v`, confirm output shows a fallback event log and a successful completion from the second provider.

After test passes: Update CLAUDE.md → "Checkpoint 1 done: LLM router built and tested with simulated 429 fallback. Cooldown default 60min. Router lives in src/leadradar/llm/router.py, providers in src/leadradar/llm/providers/."

Checkpoint 2 — Business Discovery Tool

Goal: Pull real restaurant data from Google Places for one Mumbai area.

Claude Code prompt:

```
Add discovery.py to the leadradar project with a function
discover_businesses(lat, lng, radius_m, category) that calls the
Google Places API (Nearby Search) and returns a list of dicts with:
name, address, website (or None), rating, phone, place_id.
Deduplicate by place_id. Add GOOGLE_PLACES_KEY to .env.example.
Add a small script run_discovery.py that calls this for Bandra, Mumbai,
category="restaurant", radius 3000m, and prints results as a table.
```

Test: Run `run_discovery.py`, manually verify 10–20 real restaurants appear with plausible data.

After test passes: Update CLAUDE.md → "Checkpoint 2 done: discovery.py working against Google Places. Bandra restaurant test returned N results, M had no website."

Checkpoint 3 — Website Audit Tool

Goal: For each discovered business, check if their website loads and capture a screenshot.

Claude Code prompt:

```
Add audit.py to leadradar with a function audit_website(url) using
Playwright (sync API) that:
- returns {"loaded": bool, "load_time_ms": int, "has_ssl": bool,
  "screenshot_path": str or None, "error": str or None}
- takes a full-page desktop screenshot saved to ./screenshots/{hash}.png
- has a 10-second timeout, treats timeout as loaded=False
If website is None in the business record, skip and set
loaded=False, error="no_website" without calling Playwright.
Wire this into run_discovery.py so each business is audited after discovery.
```

Test: Run against the Checkpoint 2 output, confirm screenshots folder fills up and no-website businesses are correctly flagged without crashing.

After test passes: Update CLAUDE.md → "Checkpoint 3 done: audit.py using Playwright, screenshots saved locally. Timeout set to 10s."

Checkpoint 4 — LLM Verdict on Screenshots

Goal: Route each screenshot + metadata through the LLM Router for a plain-English verdict.

Claude Code prompt:

```
Add verdict.py to leadradar with a function get_verdict(business, audit_result)
that builds a prompt describing the business + audit signals (loaded,
load_time_ms, has_ssl, no_website flag) and, if a screenshot exists,
includes it as an image in the LLM call via the Router (use a
vision-capable model). Ask for a JSON response with:
{"needs_redesign": bool, "reasoning": "2-3 sentences"}.
Parse and validate this JSON, retry once on parse failure.
Wire into run_discovery.py after the audit step.
```

Test: Run full pipeline on 10 businesses, manually spot-check 3 verdicts for sanity (do they match what you see in the screenshot?).

After test passes: Update CLAUDE.md → "Checkpoint 4 done: verdict.py gets LLM opinion per business via Router. Spot-checked N/10 verdicts, quality notes: …"

Checkpoint 5 — Scoring + Final JSON Output

Goal: Combine everything into a scored, exportable lead list.

Claude Code prompt:

```
Add scoring.py to leadradar with score_lead(business, audit_result, verdict)
implementing the FR-4 scoring rules from the SRS (no-website +40,
needs_redesign +30, low rating +10, no ssl +10, slow load +10),
returning {"score": int, "bucket": "HOT"|"WARM"|"COLD"}.
Update run_discovery.py to run the full pipeline end-to-end
(discover -> audit -> verdict -> score) and write results to
./output/leads_{timestamp}.json, plus print a summary table
(name, bucket, score, needs_redesign) to console.
```

Test: Full end-to-end run on Bandra restaurants, confirm JSON file is well-formed and the console summary makes intuitive sense (worst websites score highest).

After test passes: Update CLAUDE.md → "Checkpoint 5 done: MVP pipeline complete end-to-end. Phase 1 goal achieved. Next: decide on Phase 2 priorities (dashboard vs multi-vertical vs outreach gen)."

---

11. Definition of Done (Phase 1 / MVP)

- ☐ Checkpoint 1–5 all complete and logged in CLAUDE.md
- ☐ One full run against real Bandra restaurant data produces a JSON lead list with scores
- ☐ LLM Router has demonstrably survived at least one real (not simulated) provider quota exhaustion during testing
- ☐ No hardcoded API keys in source files
- ☐ You can look at the HOT-bucket leads and agree, by eye, that they're genuinely bad websites